

ALU Control

ALU Control Lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	Set on less than
1100	NOR

- We can generate the 4 bit ALU control input using a small control unit that has as inputs the function field of the instruction and the two bit control field, which is ALUOp.
- ALUOp indicates whether the operation to be performed should be add(00) for loads and stores, subtract(01) for beq, or determined by the operation encoded in the function field(10).
- The output of the ALU control unit is a 4-bit signal that directly controls the ALU by generating one of the 4-bit combinations.

How to set the ALU control inputs based on the 2-bit ALUOp control and the 6-bit function code

Instruction opcode	ALUOp	Instruction operation	Function Field	Desired ALU action	ALU control input
Lw	00	Load word	xxxxxx	add	0010
Sw	00	Store word	xxxxxx	add	0010
Branch equal	01	Branch equal	xxxxxx	subtract	0110
R-type	10	Add	100000	add	0010
R-type	10	Subtract	100010	subtract	0110
R-type	10	AND	100100	AND	0000
R-type	10	OR	100101	OR	0001
R-type	10	Set on less than	101010	Set on less than	0111

- The main control unit generates the ALUOp bits, which then are used as input to the ALU control that generates the actual signals to control the ALU unit.
- Using multiple levels of control can reduce the size of the main control unit.
- Using several smaller control units may also potentially increase the speed of the control unit.

Truth table for three ALU control bits

ALUOp1	ALUOp2	Function Field						operation
		F5	F4	F3	F2	F1	F0	
0	0	X	x	x	x	x	x	0010
X	1	X x x x x x						0110
1	X	100000						0010
1	X	100010						0110
1	X	100100						0000
1	X	100101						0001
1	X	101010						0111

Designing the Main Control Unit

- To connect the fields of an instruction to the datapath ,the instruction formats must be reviewed.
- Op field:31-26 bits
- Two registers to be read rs and rt :25-21,20-16.true for R-type,branch equal and for store.
- The base register for load and store instructions is always in bit positions 25-21(rs).
- R-type:

Op	Rs	Rt	Rd	Shamt	Funct
31-26	25-21	20-16	15-11	10-6	5-0

Load or store instruction

35 or 43	Rs	Rt	Address
31-26	25-21	20-16	15-0

load:35,store:43
branch instruction

4	Rs	Rt	Address
31-26	25-21	20-16	15-0

Field	0	rs	rt	rd	shamt	funct
Bit positions	31:26	25:21	20:16	15:11	10:6	5:0

a. R-type instruction

Field	35 or 43	rs	rt	address
Bit positions	31:26	25:21	20:16	15:0

b. Load or store instruction

Field	4	rs	rt	address
Bit positions	31:26	25:21	20:16	15:0

c. Branch instruction

Simple datapath with control unit

- The input to the control unit is the 6-bit opcode field from the instruction. The outputs of the control unit consists of three 1-bit signals that are used to control multiplexors(RegDst,ALUSrc and MemtoReg),
- Three signals for controlling the reads and writes in register file and data memory(RegWrite,MemRead,MemWrite).
- A 1-bit signal used in determining whether to possibly branch(Branch),and a two bit control signal for ALU(ALUOp).
- An AND gate is used to combine the branch control signal and the zero output from the ALU.
- The AND gate output controls the selection of the next PC

- The setting of the control lines is completely determined by the opcode fields of the instruction

Instruction	RegDst	ALUSrc	MemtoReg	RegWrite	Memread	MemWrite	Branch	ALUOp1	ALUOp0
R-format	1	0	0	1	0	0	0	1	0
Lw	0	1	1	1	1	0	0	0	0
Sw	X	1	X	0	0	1	0	0	0
beq	X	0	X	0	0	0	1	0	1

- The first row: R-type(add,sub and,or,slt).For all these instructions,the source register fields are rs and rt and the destination rd.this define how the signals ALUSrc and RegDst are set.

- An R-type instruction writes a register($\text{RegWrite}=1$),but neither reads nor writes data memory.
- When the branch control signal is 0,the PC is unconditionally replaced with $\text{PC}+4$.Otherwise the PC is replaced by branch target if the zero output of ALU is also high.
- The ALUOp field of R-type instruction is set to 10 to indicate the ALU control should be generated from the function field.
- The second and third row shows control signal settings for lw and sw.
- These ALUSrc and ALUOp fields are set to perform the address calculation.
- The MemRead and MemWrite are set to perform memory access.
- Finally the RegDst and RegWrite are set for a load to cause the result to be stored into the rt register.

- The branch instruction is similar to R-type, since it sends the rs and rt registers to the ALU.
- The ALUOp field for branch is set for a subtract (ALU Control=01), which is used to test for equality.
- Note: MemtoReg field is irrelevant when the RegWrite Signal is 0, since the register is not being written, the value of the data on the register data write port is not used.
- Thus the entry MemtoReg in the last two rows of the table is replaced with X for don't care.
- Don't care can also be added to RegDst when RegWrite is 0, this type of don't care must be added by the designer, since it depends on knowledge of how the datapath works.

Finalizing the control: The control function for single cycle implementation specified by truth table

Input or output	Signal name	R-Format	Lw	Sw	beq
Inputs	Op5	0	1	1	0
	Op4	0	0	0	0
	Op3	0	0	1	0
	Op2	0	0	0	1
	Op1	0	1	1	0
	Op0	0	1	1	0
Outputs	Regdst	1	0	X	X
	ALUSrc	0	1	1	0
	MemtoReg	0	1	X	X
	RegWrite	1	1	0	0
	MemRead	0	1	0	0
	MemWrite	0	0	1	0
	Branch	0	0	0	1
	ALUOp1	1	0	0	0
	ALUOp0	0	0	0	1

Implementing jumps

- The jump instruction look somewhat like branch instruction but computes the target PC and is not conditional.
- Like a branch the lower order two bits of jump address are always 00, the next lower 26 bits of this 32 bit address come from the 16 bit immediate field in the instruction



- The upper 4 bits of the address that should replace the PC come from the PC of the jump instruction plus 4, we can implement the jump by storing into the PC the concatenation of:
 - -the upper 4 bits of the current PC+4(31:28)
 - -The 26bit immediate field of jump instruction
 - -The bit 00

Why single cycle implementation is not used today?

- All though the single cycle design will work correctly, it would not be used in modern designs because it is inefficient. The clock cycle must have the same length for every instruction in this single cycle ,the CPI therefore is 1.
- The clock cycle is determined by the longest possible path in the machine.
- This path is almost the load instruction, which uses five functional units in series, *the instruction memory, register file, the ALU, the data memory and the register file.*
- Although the CPI is 1,the overall performance of single cycle implementation is not likely to be very good, since several of the instruction classes could fit in a shorter clock cycle.

- Unfortunately implementing the variable speed clock for each instruction class is extremely difficult, an alternative is to use a shorter clock cycle that does less work and then vary the number of clock cycles for the different instruction classes.
- The single cycle implementation violates the design principle of making the common case fast.
- In this single cycle implementation each functional unit can be used only once per clock; therefore some functional units must be duplicated raising the cost of implementation.
- Hence inefficient in its performance and in its hardware cost.
- These difficulties can be avoided by using shorter clock cycle-derived from the basic functional unit delays, and that requires multiple clock cycles for each instruction.

Multicycle implementation

- Also called as multiple clock cycle implementation. An implementation in which an instruction is executed in multiple clock cycles.
- The multicycle implementation allows a functional unit to be used more than once per instruction as long as it is used on different clock cycles.
- The sharing can help reduce the amount of hardware required.
- The ability to allow instructions to take different numbers of clock cycles and the ability to share functional units within the execution of a single instruction are the major advantages of a multi cycle design

Difference with single cycle version

- A single memory unit is used for both instructions and data.
- There is a single ALU, rather than ALU and two adders.
- One or more registers are added after every major functional unit to hold the output of that unit until the value is used in a subsequent clock cycle.
- At the end of the clock cycle, all data that is used in subsequent clock cycles must be stored in a state element.
- Data used by *subsequent instructions* in a later clock cycle is stored into one of the programmer-visible state elements: register file, PC or the memory.
- Data used by same instruction in a later cycle must be stored into one of these additional registers.

- The position of additional registers is determined by two factors:
What combinational unit will fit in one clock cycle and what data are needed in later cycles implementing the instruction.
- In the multicycle design it is assumed that the clock cycle can accommodate at most one of the following operations: ***A memory access, a register file access(two reads or one write) or an ALU operation.***
- Any data produced by one of these functional units must be saved into a temporary registers for use on later cycle
- If we are not saved then the possibility of timing race could occur,leading to then use of incorrect value.
- All the registers except IR hold data only between a pair of adjacent clock cycles and will thus not need a write control signal.
- Thus IR needs to hold instruction until the end of the execution of that instruction and thus will require a write control signal